

Labo 00 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde stramien: het antwoord zelf, het mechanisme erachter, een nuance of valkuil, en een voorbeeld dat je zelf aan de terminal kunt controleren.

Vraag 1 — Een Linux-binary draait overal... behalve op Windows

Antwoord. De binary vraagt niets aan "Ubuntu" of "Alpine". Hij vraagt alles aan de **kernel**, via system calls (`open`, `read`, `fork` ...), en die calls zijn op elke Linux-machine identiek. De distributie levert alleen de userland eromheen. De Windows-kernel biedt een andere, incompatibele set calls — daar vindt de binary dus geen aanspreekpunt.

Waarom. De interface tussen kernel en programma's is stabiel en gestandaardiseerd; Linux maakt er een erezaak van om ze nooit te breken. Alles wat distributies van elkaar onderscheidt — pakketbeheerder, bibliotheekversies, configuratie — zit *boven* die interface.

Nuance. "Identiek" veronderstelt wel dat de nodige gedeelde bibliotheken aanwezig zijn. De `libc` bijvoorbeeld verschilt tussen Debian en Alpine — daar bots je op in labo 05. En WSL "vertaalt" niets: WSL 2 draait een **echte** Linux-kernel in een VM.

Voorbeeld.

```
uname -r          # 6.6.87.2-microsoft-standard-WSL2: een echte Linux-kernel, gebouwd door
                  Microsoft
uname -m          # x86_64: de architectuur – de andere compatibiliteitsvoorwaarde
```

Vraag 2 — De wees die PID 1 adopteert

Antwoord. De shell — de ouder van die `sleep` — stierf samen met de terminal. De kernel laat een proces nooit zonder ouder achter: PID 1 (`systemd`) **adopteert** de wees, en daarom staat er nu `1` als PPID. De `sleep` zelf draait gewoon verder.

Waarom. Een ouder heeft een specifieke taak: wanneer een kind sterft, haalt de ouder de exitcode op (tot dan blijft het kind hangen als "zombie"). Het systeem heeft dus altijd een voogd van laatste toevlucht nodig — een van de taken van PID 1.

Nuance. Precies daarom is de PID 1 *van een container* een ernstig onderwerp (labo 03): daar erft jouw applicatie die voogdijrol zonder het te beseffen. Een PID 1 die zijn zombies nooit opruimt, of geen reserve-ouder achter zich heeft, verandert het gedrag van de container.

Voorbeeld.

```
sleep 300 &
ps -o pid,ppid,cmd | grep [s]leep    # 2419 2363 sleep 300 (PPID = jouw bash)
# sluit de terminal, open een nieuwe:
ps -ef | grep [s]sleep                # ubuntu 2419 1 ... sleep 300 (PPID = 1)
```

Vraag 3 — Permission denied, code 126

Antwoord. Het bestand mist de uitvoerbit `x`. Controleer met `ls -l deploy.sh`: je ziet `-rw-r--r--`, nergens een `x`. Oplossen doe je met `chmod +x deploy.sh`. En de omweg: `bash deploy.sh`. Dan

start je `bash` (dat zelf wél uitvoerbaar is) en krijgt het script de rol van gewoon argument dat wordt voorgelezen.

Waarom. `./deploy.sh` vraagt de kernel om **dit bestand uit te voeren**; de kernel controleert de `x`-bit en weigert. Code 126 is de shellafpraak voor "gevonden, maar niet uitvoerbaar" — niet te verwarren met 127, "niet gevonden".

Nuance. Een bestand dat je met een editor aanmaakt of downloadt, begint als `rw-`: uitvoeren is een recht dat je bewust toekent. In een Dockerfile (labo 04) faalt een `COPY` van een script gevolgd door `RUN ./script.sh` op exact dezelfde manier als de `x`-bit in je Git-repository ontbrak.

Voorbeeld.

```
printf '#!/bin/bash\nnecho hallo\n' > deploy.sh
./deploy.sh ; echo $?      # bash: ./deploy.sh: Permission denied ; 126
chmod +x deploy.sh
./deploy.sh                # hallo
```

Vraag 4 — Shellvariabele versus omgevingsvariabele

Antwoord. Regel 1 drukt `1:` af (niets). Regel 2 drukt `2: hallo` af. Vóór de `export` bestaat `MSG` alleen in **de huidige shell**; het kind `bash -c` start met de geërfde omgeving, en daar zit `MSG` niet in. Na `export` hoort `MSG` bij de omgeving, en elk kind krijgt er een kopie van.

Waarom. Bij zijn geboorte krijgt een proces een **kopie** van de omgeving van zijn ouder — nooit een referentie. De erfenis stroomt in één richting en ligt vast op het moment van de start.

Nuance. De **enkele** aanhalingstekens in `'echo 1: $MSG'` zijn essentieel: ze beletten dat jouw eigen shell `$MSG` al invult voor het kind start. Met dubbele aanhalingstekens zouden beide regels `hallo` tonen — maar dan had de ouder de invulling gedaan, niet het kind. Belangrijk gevolg: de omgeving van een proces dat **al draait**, kun je niet meer veranderen. Daarom legt `podman run -e` alles vast bij de start (labo 08).

Voorbeeld.

```
MSG=hallo
env | grep MSG      # niets
export MSG
env | grep MSG      # MSG=hallo
```

Vraag 5 — Code 137

Antwoord. $137 = 128 + 9$: het proces stierf door signaal nummer 9, `SIGKILL`. Het kreeg geen enkele kans om netjes af te sluiten. De code is berucht omdat een neergehaalde container hem achterlaat — na een `docker kill`, na een `docker stop` waarvan de gratieperiode verstreek, of nadat de **OOM killer** van de kernel ingreep omdat het geheugen op was.

Waarom. De shell codeert dood-door-signaal als `128 + signaalnummer`, om ze te onderscheiden van een vrijwillige `exit n`. En `SIGKILL` wordt nooit echt bij het proces afgeleverd: de kernel veegt het ter plekke weg, zonder ook maar één regel opruimcode uit te voeren.

Nuance. Een 137 onderzoeken betekent dus uitzoeken **wie** de 9 stuurde: een mens, een orchestrator, of de kernel zelf. `dmesg | grep -i "out of memory"` beslecht het OOM-geval. In labo 03 voer je deze diagnose uit op echte containers.

Voorbeeld.

```
bash -c 'kill -9 $$' ; echo $?    # 137 ($$ = de PID van de kind-bash zelf)
sleep 300 & kill -9 $! ; wait $! ; echo $?    # ook 137
```

Vraag 6 — Beleefde `kill`, brute `kill -9`

Antwoord. Een gewone `kill` stuurt `SIGTERM`, en dat is een **verzoek**: het proces ontvangt het, mag zijn afsluitcode uitvoeren — buffers wegschrijven, transacties afronden, verbindingen sluiten — en stopt dan. `kill -9` stuurt `SIGKILL`, dat helemaal niet wordt afgeleverd: de kernel veegt het proces meteen weg. Een databank verliest dan alles wat de schijf nog niet had bereikt, en moet bij de herstart haar journaal herspelen — of erger, beschadigde bestanden herstellen.

Waarom. Dit is precies wat `docker stop` doet: het stuurt `SIGTERM` naar PID 1 van de container, wacht een gratieperiode af (standaard 10 seconden) en stuurt dan `SIGKILL` als het proces niet gehoorzaamde. Een applicatie die `SIGTERM` negeert, sterft dus **altijd** op de harde manier zodra de termijn verstrijkt.

Nuance. `kill -9` heeft zijn plaats — voor een vastgelopen proces dat `SIGTERM` echt negeert. De juiste gewoonte is escaleren: `kill`, wachten, en pas daarna `kill -9`. Nooit omgekeerd. En noteer: `SIGKILL` kan niet worden opgevangen of genegeerd, wat het tot het enige gegarandeerde redmiddel maakt.

Voorbeeld.

```
sleep 300 &
kill %1      # SIGTERM: de job meldt "Terminated"
sleep 300 &
kill -9 %1   # SIGKILL: de job meldt "Killed"
```

Vraag 7 — `-rw-r----- root shadow` lezen

Antwoord. De negen bits vallen uiteen in drie tripletten: eigenaar `rw-`, groep `r--`, anderen `--`. Jouw gebruiker is geen `root` (de eigenaar) en zit niet in de groep `shadow`, dus geldt het triplet "anderen" — en dat geeft niets. Vandaar de weigering. `sudo cat` draait `cat` met UID 0, en voor `root` slaat de kernel de permissiecontroles gewoon over. Zonder `sudo` kunnen alleen `root` en de leden van de groep `shadow` (alleen lezen) het bestand openen.

Waarom. Bij elke `open` vergelijkt de kernel de UID/GID van het aanroepende **proces** met de bits van het bestand: eerst eigenaar, dan groep, dan anderen. Het eerste triplet dat van toepassing is, is meteen het enige dat telt.

Nuance. `/etc/shadow` bevat de wachtwoordhashes — het schoolvoorbeeld bij uitstek. En de regel "eerste toepasselijke triplet" kan verrassen: een bestand met permissies `---rw-rw-` zou onleesbaar zijn... voor zijn eigen eigenaar.

Voorbeeld.

```
id                # uid=1000(ubuntu) ...: geen root, niet in groep shadow
cat /etc/shadow   # Permission denied, code 1
sudo head -n 1 /etc/shadow # root*:20501:0:99999:7:::
```

Vraag 8 — Twee stromen, twee bestanden

Antwoord. Het scherm toont **niets**. In `resultaat.txt` staat de succesregel (`/etc/hostname`); in `fouten.txt` staat `ls: cannot access '/onbestaand-pad': No such file or directory`. Met `>` `resultaat.txt 2>&1` zouden beide regels in `resultaat.txt` belanden en zou `fouten.txt` niet eens worden aangemaakt.

Waarom. `ls` schrijft zijn resultaten naar **stdout** (stroom 1) en zijn klachten naar **stderr** (stroom 2). `>` leidt alleen stroom 1 om, `2>` alleen stroom 2. `2>&1` betekent: "laat stroom 2 wijzen naar waar stroom 1 *op dit moment* naartoe wijst".

Nuance. De volgorde telt: `2>&1 > resultaat.txt` zou de fouten naar het scherm sturen, want stroom 2 wordt vastgeklemd aan de oude bestemming van stroom 1, vóór de omleiding. Dankzij deze scheiding van stromen kan `podman logs` je straks zowel de fouten als de gewone uitvoer van een container tonen (labo 03).

Voorbeeld.

```
ls /etc/hostname /onbestaand-pad > resultaat.txt 2> fouten.txt
cat resultaat.txt      # /etc/hostname
cat fouten.txt         # ls: cannot access '/onbestaand-pad': No such file or directory
```

Vraag 9 — 127.0.0.1 versus 0.0.0.0

Antwoord. Redis luistert op `127.0.0.1:6379` — alleen de *loopback*-interface — en is dus **enkel vanaf de machine zelf** bereikbaar. Java luistert op `0.0.0.0:8080` — alle interfaces — en is dus vanaf het netwerk bereikbaar. Vanaf een andere machine antwoordt alleen de Java-dienst.

Waarom. Het luisteradres werkt als een filter: de kernel geeft het proces alleen de verbindingen door die op dat adres binnenkwamen. `0.0.0.0` betekent "elk adres dat deze machine heeft".

Nuance. Dit is een beveiligingsgrens van formaat: een databank die lokaal luistert, valt vanaf het netwerk simpelweg niet aan te vallen. In labo 07 zie je dat `podman run -p 8080:80` standaard op `0.0.0.0` publiceert, en dat `-p 127.0.0.1:8080:80` dat bewust inperkt. Wie deze twee `ss`-regels snapt, snapt `-p` eigenlijk al.

Voorbeeld.

```
python3 -m http.server 8080 --bind 127.0.0.1 &
ss -tlnp | grep 8080      # LISTEN ... 127.0.0.1:8080 ... ("python3",pid=...)
kill %1
```

Vraag 10 — Poort 80 geweigerd

Antwoord. Poorten **onder 1024** — de zogenaamde *geprivilegieerde* poorten — kan alleen root (UID 0) openen. Jouw proces draait als UID 1000, dus weigert de kernel zijn `bind` op poort 80; 8080 ligt boven de drempel en staat voor iedereen open. Historisch garandeerde die regel dat op een gedeelde machine geen gewone gebruiker zich kon voordoen als een "officiële" dienst (poort 25, poort 80...). Het gevolg vandaag: rootless Podman is een gewoon proces onder jouw UID en kan dus geen `-p 80:80` publiceren — je publiceert `-p 8080:80` in de plaats.

Waarom. De kernel voert de controle uit tijdens de system call `bind`, op basis van de effectieve UID (strikt genomen op basis van een *capability* die root bezit).

Nuance. De drempel is instelbaar (`sysctl net.ipv4.ip_unprivileged_port_start`), en echte productiewebserveren zitten achter een load balancer die poort 80 voor hen bezet. De Podman-variant van deze fout (`pasta failed ... Permission denied`) komt terug in labo 07.

Voorbeeld.

```
python3 -m http.server 80
# PermissionError: [Errno 13] Permission denied
python3 -m http.server 8080 & # werkt
kill %1
```

Vraag 11 — `/proc`, de map die er niet echt is

Antwoord. `/proc` is een **virtueel bestandssysteem** (type `proc`), gemount op `/proc`. De inhoud staat op geen enkele schijf: de kernel fabriceert elke lezing ter plekke, op basis van zijn interne toestand. De genummerde mappen zijn de levende processen; de overige bestanden beschrijven het systeem. Typische toepassingen: `/proc/<pid>/environ` (de echte omgeving van een proces), `/proc/meminfo` (het geheugen), `/proc/self/uid_map` (de UID-toewijzingen — jouw bewijs van rootless in labo 01).

Waarom. "Alles is een bestand": door de kerneltoestand als bestanden te tonen, worden `cat`, `grep` en `ls` beheergereedschap, zonder aparte API. `ps` is niets meer dan een nette voorkant voor `/proc`.

Nuance. Daarom krijgt elke container bij zijn geboorte **een eigen** `/proc` gemount — anders zou hij alle processen van de host zien. Lijkt `ps` te liegen in een container, dan spreekt daar de geïsoleerde `/proc`, niet een systeem waar processen uit verdwenen zijn.

Voorbeeld.

```
findmnt -t proc # /proc proc proc rw,relatime
df -h /proc 2>/dev/null # geen schijf achter
tr '\0' '\n' < /proc/self/environ | head -3 # de omgeving... van deze cat zelf
```

Vraag 12 — `command not found`, code 127

Antwoord. De shell liep alle mappen uit `PATH` af, in volgorde, op zoek naar een uitvoerbaar bestand met de naam `mijntool`. `~/tools` staat niet op die lijst, dus mislukte de zoektocht met code 127. De noodoplossing: een expliciet pad gebruiken, `~/tools/mijntool`. Blijvend: (1) de map toevoegen aan het `PATH` in `~/.bashrc` (`export PATH="$HOME/tools:$PATH"`), of (2) de tool kopiëren of linken naar een map die al op de lijst staat, zoals `~/local/bin` of `/usr/local/bin`.

Waarom. Het `PATH` is de enige manier waarop de shell "kale" commandonamen opzoekt. De huidige map staat er met opzet niet in: een vervalste `ls` die iemand in `/tmp` achterliet, mag niet draaien alleen omdat jij toevallig `cd /tmp` deed.

Nuance. Hou 127 (niet gevonden) en 126 (gevonden maar niet uitvoerbaar) goed uit elkaar — het zijn twee verschillende diagnoses. In een container heeft de fout `exec: "mijntool": executable file not found in $PATH` exact dezelfde oorzaak: het `PATH` van de image (labo 04).

Voorbeeld.

```
mkdir -p ~/tools && printf '#!/bin/bash\nnecho ok\n' > ~/tools/mijntool && chmod +x  
~/tools/mijntool  
mijntool                # command not found ; echo $? → 127  
export PATH="$HOME/tools:$PATH"  
mijntool                # ok
```

Vraag 13 — Waarom 12-factor zo van de omgeving houdt

Antwoord. Omdat de omgeving bij het **proces** hoort, niet bij de machine: ze wordt vastgelegd bij de start, automatisch geërfd, en verdwijnt samen met het proces. Voor wegwerpapplicaties die je zo herstart, levert dat configuratie op die (1) van buitenaf kan worden ingespoten zonder aan code of geleverde bestanden te raken, (2) per instantie kan verschillen — twee processen naast elkaar, twee configuraties — en (3) geen sporen nalaat: herstart met andere waarden, en er valt niets op te ruimen.

Waarom. De overerving van ouder naar kind doet al het werk. Wie het proces start — de shell, systemd, straks de containerengine — stelt het woordenboek samen; de applicatie hoeft alleen te lezen. Hetzelfde artefact, JAR of image, reist ongewijzigd van de ene omgeving naar de andere; alleen de set variabelen verandert.

Nuance. Dat de erfenis bevroren is, is meteen ook de beperking: een variabele wijzigen betekent het proces **herstarten**. Voor een container — wegwerpbaar van bij het ontwerp — kost dat niets; wie hoopte om een draaiend proces te herconfigureren, vangt bot. En geheimen verdienen straks beter dan variabelen die je in `/proc/<pid>/environ` kunt nalezen (labo 08).

Voorbeeld.

```
SERVER_PORT=9090 java -jar app.jar    # dezelfde JAR, een andere poort – niets werd gewijzigd  
# later wordt dat, woord voor woord:  
# podman run -e SERVER_PORT=9090 mijn-api
```